

OPHR: Mastering Volatility Trading with Multi-Agent Deep Reinforcement Learning

Research Paper Analysis & Implementation Report

Course: CT-469 Reinforcement Learning | Spring 2026

Course Instructor: Dr. Murk Marvi

Group Members:

Name	Roll No
Syed Hussain Raza	AI-20036
Syed Muhammad Bilal Hussain	AI-22046

Paper: "OPHR: Mastering Volatility Trading with Multi-Agent Deep Reinforcement Learning"

Venue: NeurIPS 2025

Authors: Zeting Chen, Xinyu Cai, Molei Qin, Bo An (Nanyang Technological University / Skywork AI)

1. Critical Paper Analysis

1.1 Problem Statement

This paper addresses a fundamental and long-standing challenge in quantitative finance: profiting from the difference between implied volatility (IV) and realized volatility (RV) in options markets. Options prices embed an IV premium, reflecting the market's expectation of future price fluctuations. When IV and RV diverge, trading opportunities arise. However, exploiting these opportunities requires solving two tightly coupled challenges simultaneously: (i) volatility timing, meaning correctly deciding when to go long volatility (buy options expecting large moves) versus short volatility (sell options expecting calm markets), and (ii) dynamic hedging, meaning managing the directional risk of option

positions through continuous delta hedging while maximizing path-dependent profits.

Traditional approaches rely on the Black-Scholes-Merton (BSM) framework, which assumes constant volatility, continuous trading, and zero transaction costs. These assumptions are systematically violated in real markets, particularly in cryptocurrency options where 24/7 trading, extreme tail events, and high volatility regimes create a complex, non-stationary environment. The paper specifically targets BTC and ETH options on the Deribit exchange, leveraging the transparency and data accessibility of crypto markets to build and evaluate a data-driven solution.

The core difficulty is that volatility trading is not a simple prediction problem. Even if one correctly forecasts that RV will exceed IV, the path the underlying price takes during the holding period determines the actual profit and loss (PnL). As the paper demonstrates in Figure 1, the same level of RV can produce vastly different outcomes depending on whether the market oscillates or trends, making hedging strategy selection a critical component of overall profitability.

1.2 Novel Contribution

The paper introduces OPHR, the first reinforcement learning framework specifically designed for volatility trading through options. While RL has been applied to stock trading, portfolio management, and high-frequency trading, options volatility trading remained untouched due to its inherent complexity involving Greeks management, path-dependent payoffs, and the need for coordinated position and hedging decisions.

The key novelties are:

1. **Multi-Agent Architecture:** OPHR decomposes the problem into two specialized cooperative agents rather than attempting a monolithic solution. The Option Position Agent (OP-Agent) handles volatility timing, deciding long/short/neutral positions, while the Hedger Routing Agent (HR-Agent) selects the optimal hedging strategy from a pool of hedgers with varying risk preferences. This decomposition mirrors how professional volatility traders actually operate, separating the "what to trade" decision from the "how to manage risk" decision.
2. **Twin Environment Training for HR-Agent:** The HR-Agent is trained using a novel relative reward mechanism. At each decision point, the framework runs the selected hedger in the main environment and the

baseline hedger in a twin (duplicate) environment for the same time window. The reward is the difference in net values, ensuring the HR-Agent learns to outperform the baseline rather than optimizing an absolute but potentially noisy signal.

3. **Oracle-Guided Initialization:** Rather than training from scratch with sparse and delayed rewards, Phase 1 uses a sub-optimal Oracle policy that has access to future RV to generate initial experience. This distillation accelerates convergence by providing the OP-Agent with a warm start that captures profitable trading behaviors, significantly reducing the exploration burden inherent in financial RL.
4. **Iterative Cooperative Training:** Phase 2 alternates between training the OP-Agent (with the HR-Agent frozen) and training the HR-Agent (with the OP-Agent frozen). This avoids the instability of simultaneous updates in multi-agent settings and captures the natural co-adaptation: a better hedger enables the OP-Agent to take more aggressive positions, and more aggressive positions require the HR-Agent to learn more sophisticated risk management.

1.3 Algorithm Analysis

The OPHR framework operates in two distinct phases:

Phase 1 -- Offline Initialization: - The Oracle policy generates trading signals by comparing future RV against current IV using the condition: if $\text{future RV} \geq \text{IV} * (1 + \beta)$, go long; if $\text{future RV} \leq \text{IV} * (1 - \beta)$, go short; otherwise neutral. Beta is a sensitivity threshold (default 0.1) that controls the aggression of the Oracle. - The Oracle uses a baseline delta-threshold hedger (threshold = 0.1) for risk management. - Experience tuples are collected into the OP-Agent's replay buffer, and the agent is then trained offline on this data using n-step TD learning. - Separately, the HR-Agent receives a warm-up phase where it learns initial hedger routing decisions.

Phase 2 -- Iterative Online Training: - In each iteration, the OP-Agent is trained for a set number of episodes using the current HR-Agent's hedging, followed by training the HR-Agent using the twin environment approach with the current OP-Agent. - The OP-Agent uses epsilon-greedy exploration with decaying epsilon, and is updated via n-step Double DQN with a target network. - The HR-Agent uses 1-step DQN where the action space is a discrete set of hedger indices from the hedger pool. - The alternating schedule is repeated for multiple iterations (default: 5), progressively improving both agents.

Agent Architecture: Both agents use feedforward Q-networks (QNetwork) with configurable hidden dimensions (default: [1024, 1024] with ReLU activations). The OP-Agent takes market features (volatility tickers and perpetual futures features) as input and outputs Q-values for 3 actions (long, neutral, short). The HR-Agent takes an extended state including market features, position information (number of options and perpetual position size), and Greeks (delta, gamma, theta, vega) and outputs Q-values over the hedger pool.

1.4 Mathematical Formulation

The framework is grounded in a Cooperative MDP formulation where the two agents share the common objective of maximizing portfolio net value.

Gamma-Theta Relationship (Eq. 2-3): The theoretical PnL of a delta-hedged option position is approximated by integrating the difference between instantaneous realized variance and implied variance, weighted by gamma:

$$PnL = \int_0^T \sigma^2 \Gamma (\sigma^2 - \sigma^2_{implied}) dt$$

In plain terms, this says: if the actual price fluctuations (RV) exceed what was priced into the option (IV), a long gamma position profits, and vice versa. The Gamma term amplifies this profit near the money, and Theta represents the cost of carrying the position over time. This relationship is fundamental because it shows volatility trading is a bet on RV vs. IV, not on price direction.

n-step TD Learning (Eq. 5): The OP-Agent uses n-step temporal difference learning to handle the delayed and noisy nature of options rewards:

$$L(\theta) = E[(\sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q_{\theta'}(s_{t+n}, \arg\max_a Q_{\theta}(s_{t+n}, a)) - Q_{\theta}(s_t, a_t))^2]$$

The n-step return aggregates rewards over n time steps before bootstrapping from the target network. This is essential for options trading because a position decision at time t may not show its true value until many hours later due to theta decay and delayed gamma realization. The Double DQN formulation (using the online network for action selection and the target network for evaluation) reduces overestimation bias.

HR-Agent Relative Reward: The reward for the HR-Agent at time t+n_hr is:

$$r_{hr} = V_{t+n_{hr}} - \hat{V}_{t+n_{hr}}$$

, where V is the net value under the selected hedger and $\hat{V}$ is the net value under the baseline hedger over the same n_{hr} -step window. This relative reward design is elegant because it removes the common component of PnL that comes from the option position itself, isolating the hedging contribution and providing a cleaner learning signal.

2. Implementation & Reproduction

2.1 Implementation Challenges

The publicly available codebase implements the full OPHR framework in PyTorch. Several significant challenges emerged during reproduction:

1. **Data Pipeline Complexity:** The environment requires synchronized hourly option chain data, perpetual futures data, and volatility ticker data in specific formats (Parquet and pickled dictionaries). The data handler must align timestamps, handle missing ticks, and manage option expiration cycles. The sample data provided covers only BTC from January to April 2024, which is a small fraction of the full 2019-2024 dataset used in the paper.
2. **Environment Fidelity:** The BaseEnv implements a realistic trading simulation including Deribit's portfolio margin system with risk matrix calculations, delta shock computations, roll shock calculations, and funding fee deductions. The margin model alone requires computing worst-case PnL across 27 scenarios (9 price moves x 3 volatility shocks) plus 8 extended scenarios, making the environment computationally expensive.
3. **Twin Environment Mechanism:** The HR-Agent training requires running two parallel environments from the same state, one with the selected hedger and one with the baseline. This is implemented via deep-copy state save/restore (save_state/restore_state in BaseEnv), which is memory-intensive and careful state management is needed to avoid leaking information between environments.
4. **Feature Engineering:** The state representation concatenates volatility ticker features and perpetual features (approximately 48 features each, yielding a 96-dimensional state for OP-Agent). The HR-Agent extends this with 2 position features and 4 Greeks features, totaling

approximately 102 dimensions. Feature alignment and normalization across different data sources required careful handling.

5. **Decimal Precision:** The entire environment uses Python's Decimal type for financial calculations to avoid floating-point errors in margin computation and PnL tracking. This introduces complexity when interfacing with PyTorch tensors, requiring explicit float/Decimal conversions throughout.

2.2 Hyperparameter Analysis

The algorithm's sensitivity to key hyperparameters is notable:

Hyperparameter	Default Value	Sensitivity	Notes
n-step (OP)	12 hours	High	Controls the trade-off between bias and variance in value estimation. Shorter n-steps introduce more bootstrap bias but less variance; longer steps capture more of the delayed options payoff but increase variance.
Oracle beta	0.1	Medium	Determines how conservative the Oracle signal is. Higher beta means fewer trades but higher conviction.
HR decision interval (n_hr)	24 hours	High	How often the HR-Agent re-evaluates its hedger selection. Too frequent causes excessive switching costs; too infrequent misses regime changes.
Epsilon decay	0.995	Medium	Controls exploration-exploitation trade-off. Financial environments benefit from longer exploration due to non-stationarity.
Hidden dims	[1024, 1024]	Low-Medium	The paper uses relatively large networks. Smaller networks may suffice given the moderate state dimensionality but could underfit complex volatility patterns.

Hyperparameter	Default Value	Sensitivity	Notes
Iterative iterations	5	Medium	More iterations allow better co-adaptation but risk overfitting.
Replay buffer size	100K (OP) / 50K (HR)	Low	Standard sizes; the key is ensuring sufficient diversity of market regimes.

2.3 Result Comparison

The paper reports OPHR achieving 33.10% total return on BTC and 44.89% on ETH during the July 2023 to July 2024 test period, with Sharpe ratios of 1.87 and 1.76 respectively. These results represent substantial outperformance over all baselines, including directional strategies (Long/Short), factor strategies (MR/MOM), and ML models (GBDT/MLP/LSTM/GARCH/DeepVol/DLOT).

With only the sample data (BTC, Jan-Apr 2024, a subset of the test period), full reproduction of the reported numbers is not possible. However, the code architecture is complete and functional, and the training pipeline can be executed end-to-end. The key insight from analyzing the code is that the results are plausible: the framework correctly implements delta-hedged straddle PnL as the optimization target, and the twin environment mechanism provides a meaningful signal for hedger selection.

Notably, even the OP-only variant (without HR-Agent, using baseline hedger) achieves 21.43% on BTC, indicating that the volatility timing component alone provides significant value, while the HR-Agent adds approximately 12 percentage points through improved hedging.

2.4 Computational Requirements

- **Hardware:** The code runs on CPU (with optional CUDA support). The paper does not require expensive GPU resources; the neural networks are moderate-sized MLPs, not deep CNNs or Transformers.
- **Training Time:** Phase 1 Oracle collection with 1000 episodes is the most time-intensive due to environment simulation. Phase 2 iterative training with 5 iterations x (200 OP + 50 HR episodes) requires approximately 1250 episodes total. On a modern CPU, each episode with the full environment simulation takes several seconds to minutes depending on episode length.

- **Memory:** The twin environment approach doubles memory usage during HR training due to state save/restore with deep copies. The replay buffers (100K + 50K transitions) require modest memory.
 - **Recommended:** Google Colab free tier with CPU is sufficient for experimentation with reduced episode counts.
-

3. Critical Evaluation

3.1 Strengths

1. **Principled Problem Decomposition:** The separation into OP-Agent and HR-Agent is well-motivated by both the financial structure of volatility trading and MARL theory. It reduces the joint action space from combinatorially large (position x hedge amount) to two manageable discrete spaces, enabling practical training.
2. **Realistic Environment:** The implementation includes Deribit's actual portfolio margin model, transaction fees, funding rates, and bid-ask spreads. Most academic RL trading papers use simplified environments that ignore these real-world frictions, making their results unrealistic. OPHR's inclusion of these details strengthens the validity of its results.
3. **Oracle Initialization Strategy:** Using future information to bootstrap training is a clever approach to the cold-start problem in financial RL. The Oracle is explicitly sub-optimal (it uses a fixed beta threshold and ignores path dependence), so the agent is not simply memorizing the Oracle but learning to improve upon it.
4. **Comprehensive Evaluation:** The paper evaluates against 10 baselines spanning three paradigms (rule-based, traditional ML, deep learning), using 8 complementary metrics that capture profit, risk-adjusted performance, risk exposure, and trade quality.
5. **Transaction Cost Accountability:** OPHR reports total transaction costs as 9.36% (BTC) and 5.75% (ETH) of PnL, confirming that the strategy's profitability is not an artifact of ignoring trading costs.

3.2 Limitations

1. **No Out-of-Sample Robustness Testing:** The paper uses a single train/test split. There is no walk-forward analysis, cross-validation across market regimes, or multiple random seeds with confidence intervals.

The authors acknowledge this, noting that "we only experienced one history," but the absence of error bars makes it difficult to assess whether the results are robust or lucky.

2. **Crypto-Only Evaluation:** Results are only demonstrated on BTC and ETH options. It is unclear whether OPHR generalizes to equity options (SPX, single-stock), commodity options, or FX options, which have fundamentally different volatility dynamics, liquidity profiles, and market microstructure.
3. **Fixed Straddle Strategy:** The framework only trades ATM straddles with fixed 1-contract size. Professional volatility traders use a wider repertoire including strangles, butterflies, calendar spreads, and dynamic position sizing. This simplification may leave significant alpha on the table.
4. **No Online Adaptation:** Once trained, the agents are deployed with fixed weights. In non-stationary financial markets, model degradation over time is a major concern. The paper does not discuss online fine-tuning, concept drift detection, or model retraining schedules.
5. **Oracle Data Leakage Risk:** While the Oracle is labeled "sub-optimal," it relies on future RV, which is never available in production. If the replay buffer from Phase 1 biases the agent toward patterns that are only apparent with foresight, the reported performance may not transfer to live trading. The paper does not ablate against random initialization to quantify the Oracle's contribution.
6. **Limited Ablation Studies:** There is no systematic hyperparameter sensitivity analysis. Questions like "how does performance vary with n-step length?" or "what is the optimal hedger pool composition?" remain unanswered.

3.3 Ethical Considerations

1. **Market Manipulation Risk:** A sophisticated RL-based options trading system, if widely deployed, could contribute to market instability. Coordinated algorithmic selling during volatility spikes could exacerbate crashes (a "flash crash" scenario).
2. **Democratization vs. Systemic Risk:** Making such tools publicly available could democratize access to institutional-grade trading strategies but also increase systemic risk if many participants adopt similar strategies, leading to crowded trades.

3. **Data Transparency:** The paper uses Deribit data, which is publicly accessible, and the authors have released their code and sample data. This transparency is commendable and sets a good standard for reproducibility in financial RL research.
4. **Regulatory Compliance:** Deploying RL-based trading systems in regulated markets requires compliance with market conduct rules. The paper does not discuss the regulatory implications of autonomous options trading agents.

3.4 Comparison with Alternatives

OPHR can be compared with several alternative approaches:

- **Deep Hedging (Buehler et al., 2019-2022):** Focuses solely on hedging, not on position entry/exit timing. OPHR builds on Deep Hedging by incorporating it as part of the hedger pool but adds the crucial volatility timing component.
- **DLOT (Tan et al., 2024):** End-to-end deep learning for options trading. Achieves modest positive returns (4.91% BTC, 1.19% ETH) but lacks adaptive hedging and RL optimization, explaining its inferior performance.
- **Traditional GARCH/DeepVol:** These are forecasting models, not trading systems. They predict RV but do not optimize the path-dependent PnL, creating a gap between forecast accuracy and trading profitability that OPHR bridges.
- **Single-Agent RL:** The OP-only baseline demonstrates that even without the HR-Agent, the OP-Agent alone outperforms all baselines, validating the RL approach for volatility timing. The HR-Agent provides significant incremental value.

4. Real-World Application Proposal

Proposed Domain: Insurance Catastrophe Bond (Cat Bond) Portfolio Management

Application: The OPHR multi-agent framework could be adapted for managing portfolios of catastrophe bonds and related insurance-linked securities (ILS), where similar volatility timing and risk management challenges exist.

Why OPHR is Suitable:

1. **Analogous Structure:** Cat bond markets exhibit a "volatility premium" analogous to the IV-RV spread in options. Investors earn a spread during calm periods (similar to short gamma) but face catastrophic losses during natural disasters (similar to tail risk from unhedged short options). The OP-Agent's volatility timing capability could identify when to increase or decrease cat bond exposure based on climate and geophysical indicators.
2. **Hedging Complexity:** Cat bond portfolios require dynamic risk management using weather derivatives, reinsurance contracts, and traditional hedging instruments. An HR-Agent could route between different hedging strategies (weather derivatives, geographic diversification, basis risk hedges) based on seasonal patterns and portfolio exposure.
3. **Multi-Agent Decomposition:** The separation between "what exposure to take" and "how to hedge that exposure" maps naturally to cat bond portfolio management, where allocation and risk management are handled by different teams in practice.

Required Modifications:

1. **State Space:** Replace options market features with climate indicators (sea surface temperatures, atmospheric indices, historical loss data), reinsurance market data, and portfolio exposure metrics.
2. **Action Space:** The OP-Agent would select exposure levels across different peril types (hurricane, earthquake, flood) rather than long/short/neutral. The HR-Agent would select from a pool of hedging instruments rather than delta hedgers.
3. **Environment:** Build a simulation environment incorporating catastrophe loss models (similar to those used by AIR or RMS) instead of the options pricing environment.
4. **Reward Function:** Adapt the reward to account for the extreme skewness of cat bond returns, potentially using CVaR-based objectives rather than simple PnL changes.
5. **Training Data:** Cat bond events are rare by nature, so the Oracle policy would need to be designed using synthetic catastrophe scenarios from climate models rather than historical future information.

This application is particularly promising because the cat bond market is growing rapidly (exceeding \$45 billion in outstanding notional), is data-rich but analytically underserved, and presents the same core challenge of

balancing premium collection against tail risk management that OPHR was designed to address.

References

1. Chen, Z., Cai, X., Qin, M., & An, B. (2025). OPHR: Mastering Volatility Trading with Multi-Agent Deep Reinforcement Learning. NeurIPS 2025.
 2. Black, F., & Scholes, M. (1973). The Pricing of Options and Corporate Liabilities. *Journal of Political Economy*.
 3. Merton, R. C. (1973). Theory of Rational Option Pricing. *The Bell Journal of Economics*.
 4. Buehler, H., et al. (2019). Deep Hedging: Hedging Derivatives Under Generic Market Frictions Using RL.
 5. Murray, P., et al. (2022). Deep Hedging: Continuous RL for Hedging across Multiple Risk Aversions. ACM ICAIF.
 6. Tan, W. L., et al. (2024). Deep Learning for Options Trading: An End-To-End Approach. ACM ICAIF.
 7. Sutton, R. S., & Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. MIT Press.
 8. Mnih, V., et al. (2015). Human-Level Control through Deep Reinforcement Learning. *Nature*.
 9. Hasselt, H. V., et al. (2016). Deep Reinforcement Learning with Double Q-Learning. *AAAI*.
 10. Sinclair, E. (2013). *Volatility Trading*. John Wiley & Sons.
-